Lehrerexemplar · Boss-Kampf 4 · Welt 4 · DS 30

Erwartungshorizont: Die Fähre von Nebelhafen

Erwartungshorizont / Bewertungsbogen für _____ Korrigiert von: _____

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten

Erwartete Prüfungsleistungen	I	II	III	BE
a) Befehlsfolge auf Stapel und Schlange; Leer-Abfrage Kernpunkte: Zeitpunkt 1: deck Lkw, Bus (oben zuerst), anleger Pkw; Zeitpunkt 2: deck Pkw, Bus, anleger Lkw, Rad; Zeitpunkt 3: deck Rad, Lkw, Pkw, Bus, anleger leer; Ausgabe „Pkw Rad“; ohne die Abfrage würde head() auf der leeren Schlange einen Laufzeitfehler auslösen (Pkw ist nicht mehr in anleger); Kurzschluss: der rechte Teil wird nur ausgewertet, wenn der linke wahr ist.	2	2		
b) Eignung von Stapel, Schlange und dynamischer Reihung beurteilen Kernpunkte: Deck der neuen Fähre: Schlange – wer zuerst auffährt, verlässt die Fähre zuerst (FIFO), hinten anstellen, vorn entnehmen; Buchungsliste: Einsortieren nach Länge und Entfernen aus der Mitte brauchen Zugriff auf beliebige Positionen; bei einer Schlange nur vorn – jede Operation erfordert vollständiges Umschichten über eine Hilfsschlange (bzw. Rotieren) mit Aufwand proportional zur Länge und Fehleranfälligkeit; Urteil: Vorschlag abzulehnen, die dynamische Reihung (insertAt/delete über den Index) ist besser geeignet.	1	1	1	
c) Implementierung: Stapel auswerten und unverändert lassen Kernpunkte: Methodenkopf public int ausfahrtBis(String kennzeichen); Hilfsstapel; Schleife mit Leer-Abfrage !deck.isEmpty(); Cast (Fahrzeug) deck.pop() und Vergleich mit equals; Zählen der Fahrzeuge oberhalb des gesuchten (oberstes → 0); Abbruch beim ersten Treffer oder Merken des ersten Treffers; vollständiges Zurückschichten vor der Rückgabe (Reihenfolge bleibt); Rückgabe −1, wenn kein Treffer; kein Laufzeitfehler bei leerem Deck.	1	4	1	
d) Struktogramm: sortiert einfügen in eine dynamische Reihung Kernpunkte: Kopf einbuchen(f: Fahrzeug): nichts; Suche der Einfügeposition i von vorn, solange Fahrzeuge mindestens so lang wie f sind (≥ – hinter gleich lange); Index-Abfrage i < buchungen.getLength() vor getItem (Kurzschluss oder Hilfsvariable); insertAt(i, f), auch i = getLength() (Anhängen); Randfälle leere Liste, f vorn, f hinten ohne Laufzeitfehler; Notation nach EH §5.	1	2	1	
e) Tracetabelle mit Stapel und Schlange; Funktionsweise erläutern Kernpunkte: Tracetabelle wie in der Musterlösung; Ergebnis anleger = DEL-S 6, OL-W 8, HB-R 5, NH-M 1, NH-Z 2; Funktionsweise: Aufteilen in lange (> grenze, Stapel) und kurze Fahrzeuge (Hilfsschlange), dann zuerst die langen, danach die kurzen wieder anstellen; lange in umgekehrter, kurze in ursprünglicher Reihenfolge; Änderung: für die langen ebenfalls eine Schlange verwenden.	1	3		
f) Fehleranalyse: delete in einer Vorwärtsschleife Kernpunkte: Ablauf: i = 2 NH-E 8 verladen, delete(2) → DEL-G 1 rückt auf Index 2; i = 3 NH-H 6 verladen; i = 4: getItem(4) bei Länge 3 → IndexOutOfBoundsException; Ursache: n wird vor der Schleife festgelegt, nach delete rücken die Nachfolger nach vorn, i wird trotzdem erhöht; Auswirkungen: Überspringen (DEL-G 1 nicht verladen) und Zugriff auf ungültigen Index; Korrektur: Länge in jeder Prüfung neu abfragen und i nur erhöhen, wenn nicht gelöscht wurde (Reihenfolge auf dem Deck bleibt).	1	1	1	
Summe	7	13	4	**/ 24**

AFB-Verteilung: I 7 BE (29 %) · II 13 BE (54 %) · III 4 BE (17 %) – Abitur-Richtwert 30 / 53 / 17 %. Zeitbudget: 24 BE · 2,5 min = 60 min.

Die gewählten Lösungsansätze und -wege müssen nicht mit den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (EH 2025) entsprechend zu bewerten. Laufzeitfehler, die nicht durch explizite Abfragen ausgeschlossen werden, führen zu Abzug (EH §4).

Auswertung und Algo Quest Lab (fließt nicht in die Note ein)

- Der Papier-Boss ist **handschriftliches Abi-Training**: Auswertung durch die Lehrkraft oder Selbst-/Peer-Korrektur nach diesem Erwartungshorizont. Punkte werden dafür nicht vergeben – Punkte entstehen nur automatisch im Algo Quest Lab.
- Papier-Compiler zu c): Handschrift unverändert abgetippt, alle 8 Tests grün – Ergebnis im Antwortbogen ankreuzen (beim 1. Versuch / nach Korrektur / nicht grün).
- **Fehlerjagd** – darauf besonders achten: In c) Leer-Abfrage vor jedem pop und Deck unverändert (Tests nichtAnBord, leeresDeck, unveraendertNachTreffer, unveraendertOhneTreffer grün), in f) Ursache erkannt und korrekt korrigiert.
- **Zusätzlich im Lab: Online-Boss „Die Postkutschenstation“** (25 min, 8 Aufgaben, Auswertung am Ende, der beste Versuch zählt, „besiegt“ ab 50 %). Punkte und Abzeichen („Die Postkutschenstation besiegt“, ggf. „Makellos“) vergibt das Lab automatisch; die Ergebnisse stehen in der Kurs-Auswertung.
- Orientierung (informativ, Abi-Skala): ab 95 % 15 P · 85 % 13 P · 75 % 11 P · 65 % 9 P · 55 % 7 P · 45 % 5 P · 40 % 4 P · 27 % 2 P. Für 24 BE: 23 BE ≈ 15 P, 18 BE ≈ 11 P, 10 BE ≈ 4 P.

Musterlösungen, Kernpunkte und typische Fehler

a) Befehlsfolge auf Stapel und Schlange; Leer-Abfrage 4 BE (I 2 · II 2 · III 0)

Musterlösung

Zustände (geprüft durch Ausführen der Befehlsfolge mit Nachbauten der Bibliotheksklassen):

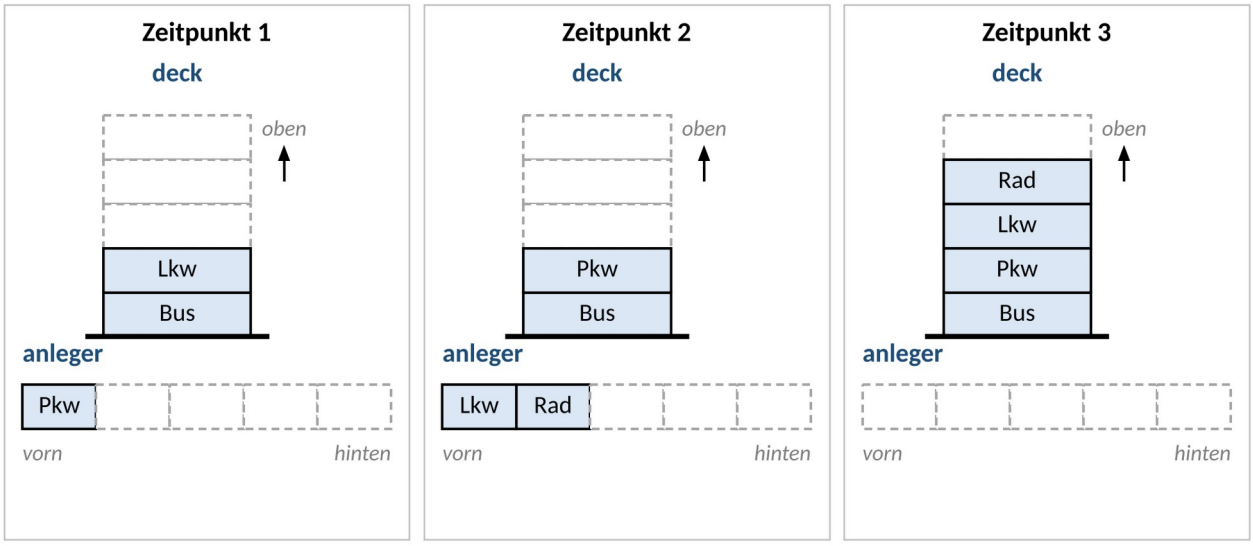


Abbildung EH.1: deck und anleger zu den Zeitpunkten 1 bis 3

Ausgabe von Zeile 16: **Pkw Rad** – x ist das oberste Element von deck nach Zeitpunkt 2 („Pkw“), danach liegt „Rad“ oben.

Zeile 13: Die Schleife legt so lange Fahrzeuge aus anleger auf deck, bis vorn „Pkw“ steht. „Pkw“ ist aber gar nicht mehr in anleger (es liegt auf deck) – die Schlange wird vollständig geleert. Ohne `!anleger.isEmpty()` würde danach `anleger.head()` auf der leeren Schlange aufgerufen → Laufzeitfehler (IllegalStateException). Weil && von links nach rechts auswertet und abbricht, sobald der linke Teil falsch ist (Kurzschlussauswertung), wird head() nur bei nicht leerer Schlange ausgeführt; in umgekehrter Reihenfolge käme der Zugriff vor der Prüfung.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Zustände zu Zeitpunkt 1 (Mischaufgabe push(dequeue()))	I	1	☐

Kernpunkt	AFB	BE	erreicht
Zustände zu Zeitpunkt 2 (pop/enqueue, Rotieren in Zeile 11)	I	1	☐
Zustand zu Zeitpunkt 3 und Ausgabe „Pkw Rad“	II	1	☐
Erläuterung: Schlange wird leer, head() auf leerer Schlange → Laufzeitfehler; Reihenfolge wegen Kurzschluss	II	1	☐

Alternativen / Teilpunkte: Zustände als Listen mit Angabe von oben/vorn sind gleichwertig zu den Skizzen. Folgefehler werden nicht erneut abgezogen. Für die Erläuterung genügt der Hinweis auf head() bei leerer Schlange und auf die Auswertung von links nach rechts.

Typische Fehler

- Turm von oben nach unten gefüllt (Bus oben).
- Zeile 10 als „Pkw bleibt in anleger“ (head statt dequeue verwechselt).
- Zeile 11 übersehen: Rotieren stellt das vorderste Element hinten an.
- „Die Schleife endet bei Pkw“ – übersehen, dass Pkw auf deck liegt.

b) Eignung von Stapel, Schlange und dynamischer Reihung beurteilen 3 BE (I 1 · II 1 · III 1)

Musterlösung

Deck der neuen Fähre: Fahrzeuge fahren hinten auf und vorn ab. Das zuerst aufgefahrene Fahrzeug steht ganz vorn und verlässt die Fähre zuerst (FIFO). Genau das leistet eine **Schlange** (enqueue hinten, dequeue vorn). Ein Stapel (LIFO) passt zur alten Fähre mit nur einer Rampe; eine dynamische Reihung wäre möglich, bietet aber nicht benötigte Zugriffe und schützt nicht vor „Vordrängeln“.

Beurteilung: Für das Verladen „der Reihe nach“ wäre eine Schlange passend. Beim Einbuchen muss aber an einer bestimmten Position eingefügt und beim Stornieren aus der Mitte entfernt werden. Eine Schlange erlaubt nur den Zugriff auf das vorderste Element; man müsste jedes Mal alle Elemente entnehmen und (mit dem neuen bzw. ohne das stornierte Fahrzeug) wieder anstellen – ein Durchlauf durch die ganze Liste pro Operation, mehr Code und mehr Fehlerquellen. Die dynamische Reihung erledigt beides direkt über den Index (insertAt, delete) und erlaubt zusätzlich die Suche über getItem. Der Vorschlag ist daher abzulehnen.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Deck der neuen Fähre: Schlange, Begründung FIFO	I	1	☐
Benötigte Zugriffe (Einfügen an Position, Entfernen aus der Mitte) vs. Zugriff nur vorn	II	1	☐
Begründetes Urteil (Umschichten der ganzen Schlange, Aufwand) zugunsten der dynamischen Reihung	III	1	☐

Alternativen / Teilpunkte: Auch ein differenziertes Urteil ist zulässig (z. B. „für eine reine Verladereihenfolge ohne Stornierungen wäre eine Schlange ausreichend“), wenn es auf die geforderten Operationen bezogen ist. Ohne Bezug auf Einsortieren/Stornieren: höchstens 1 BE für den zweiten Teil.

Typische Fehler

- „Stapel, weil die Fahrzeuge aufeinander stehen“ – Rampenlogik nicht beachtet.
- Urteil ohne Begründung über die Zugriffsmöglichkeiten.
- Behauptung, mit einer Schlange sei Einfügen in der Mitte unmöglich (es geht – aber nur durch Umschichten).

c) Implementierung: Stapel auswerten und unverändert lassen 6 BE (I 1 · II 4 · III 1)

Musterlösung

Musterlösung (Java, mit javac und den Papier-Compiler-Tests geprüft: 8/8 grün; Abbildung 2: OL-B 9 → 3, NH-K 7 → 0, NH-T 3 → 4, HB-X 1 → −1, Deck danach unverändert):

```
public int ausfahrtBis(String kennzeichen) {
    Stack hilf = new Stack();
    int anzahl = 0;
    int ergebnis = -1;
    while (!deck.isEmpty() && ergebnis == -1) {
        Fahrzeug f = (Fahrzeug) deck.pop();
        hilf.push(f);
        if (f.getKennzeichen().equals(kennzeichen)) {
            ergebnis = anzahl;
        }
        anzahl++;
    }
    while (!hilf.isEmpty()) {
        deck.push(hilf.pop());
    }
    return ergebnis;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf; Rückgabe einer Ganzzahl in jedem Pfad, −1 ohne Treffer	I	1	☐
Umschichten in einen Hilfsstapel mit Leer-Abfrage vor jedem pop	II	1	☐
Cast (Fahrzeug) und Vergleich der Kennzeichen mit equals	II	1	☐
richtige Zählung (Fahrzeuge oberhalb; oberstes → 0), erster Treffer	II	1	☐
Zurückschichten vor der Rückgabe – Deck unverändert	II	1	☐
Randfälle (leeres Deck, kein Treffer, Treffer ganz unten) ohne Laufzeitfehler, kein vorzeitiges return	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (beide ebenfalls 8/8 Tests grün): (1) mit top() prüfen und nur die Fahrzeuge oberhalb umschichten (Wahrheitswert gefunden); (2) immer den ganzen Stapel umschichten und den ersten Treffer merken. Eine Hilfsschlange ist nur richtig, wenn danach die Reihenfolge wiederhergestellt wird (z. B. Schlange → Stapel → Hilfsstapel → Stapel); einfaches Zurückschichten aus einer Schlange kehrt das Deck um: höchstens 4 BE. Fehlende Casts oder == statt equals: je 1 BE Abzug höchstens einmal; im Papier-Compiler fallen fehlende Casts als Compilerfehler auf.

Alternative (1): top() prüfen, nur Fahrzeuge oberhalb umschichten

```
public int ausfahrtBis(String kennzeichen) {
    Stack hilf = new Stack();
    boolean gefunden = false;
    int anzahl = 0;
    while (!deck.isEmpty() && !gefunden) {
        Fahrzeug f = (Fahrzeug) deck.top();
        if (f.getKennzeichen().equals(kennzeichen)) {
            gefunden = true;
        } else {
            hilf.push(deck.pop());
            anzahl = anzahl + 1;
        }
    }
    while (!hilf.isEmpty()) {
        deck.push(hilf.pop());
    }
    if (gefunden) {
        return anzahl;
    }
    return -1;
}
```

Alternative (2): ganzen Stapel umschichten, ersten Treffer merken

```
public int ausfahrtBis(String kennzeichen) {
    Stack hilf = new Stack();
    int position = 0;
    int ergebnis = -1;
    while (!deck.isEmpty()) {
        Fahrzeug f = (Fahrzeug) deck.pop();
        if (ergebnis == -1 && f.getKennzeichen().equals(kennzeichen)) {
            ergebnis = position;
        }
        position++;
        hilf.push(f);
    }
    while (!hilf.isEmpty()) {
        deck.push(hilf.pop());
    }
    return ergebnis;
}
```

Typische Fehler

- while (true) bzw. Bedingung nur auf den Treffer → pop auf leerem Stapel, wenn das Kennzeichen fehlt (Tests nichtAnBord, leeresDeck rot).
- return im Schleifenrumpf beim Treffer → Deck nicht wiederhergestellt (Test unveraendertNachTreffer rot).
- Zurückschichten aus einer Hilfsschlange → Reihenfolge umgekehrt.
- Gesuchtes Fahrzeug mitgezählt (OL-B 9 → 4).
- Kein Cast: Object hat keine Methode getKennzeichen (Compilerfehler) bzw. Vergleich mit ==.
- Deck durch `deck = hilf;` „wiederhergestellt“ – Reihenfolge umgekehrt bzw. nicht umgeschichtete Fahrzeuge verloren.

d) Struktogramm: sortiert einfügen in eine dynamische Reihung 4 BE (I 1 · II 2 · III 1)

Musterlösung

Musterlösung (in Java übersetzt und geprüft: Beispiel NH-N 9 (7 m) → Index 4; Einfügen ganz vorn und ganz hinten; 4000 Zufallsfälle gegen stabiles Sortieren):

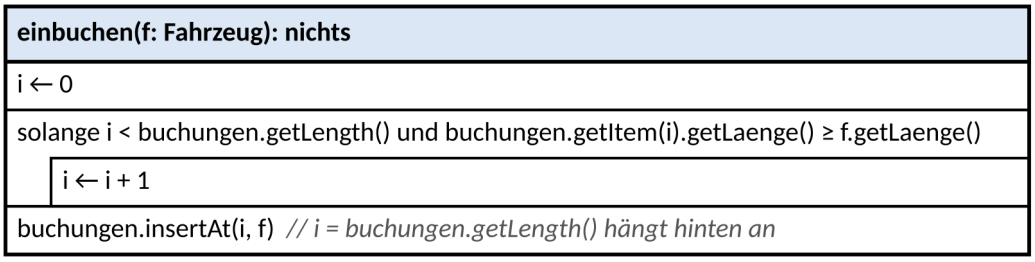


Abbildung EH.2: Struktogramm der Operation einbuchen

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Kopf mit Signatur, Zählvariable und insertAt	I	1	☐
Suche der Einfügeposition mit getItem(i).getLaenge() und ≥ (hinter gleich langen)	II	1	☐
Index-Abfrage vor getItem (kein Zugriff mit i = getLength())	II	1	☐
Randfälle (leer, ganz vorn, ganz hinten) korrekt, keine Doppel- oder Fehleinfügung	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: Variante mit Wahrheitswert eingefuegt (Abbildung EH.3), die nicht auf die Kurzschlussauswertung angewiesen ist, ebenfalls geprüft. Auch eine Zählschleife „für i von 0 bis buchungen.getLength() − 1“ mit Einfügen beim ersten kürzeren Fahrzeug ist richtig, wenn danach nicht weiter eingefügt wird und der Fall „anhängen“ behandelt ist. > statt ≥ (f vor gleich lange Fahrzeuge): 1 BE Abzug. Java-Code statt Struktogramm: höchstens 2 BE.

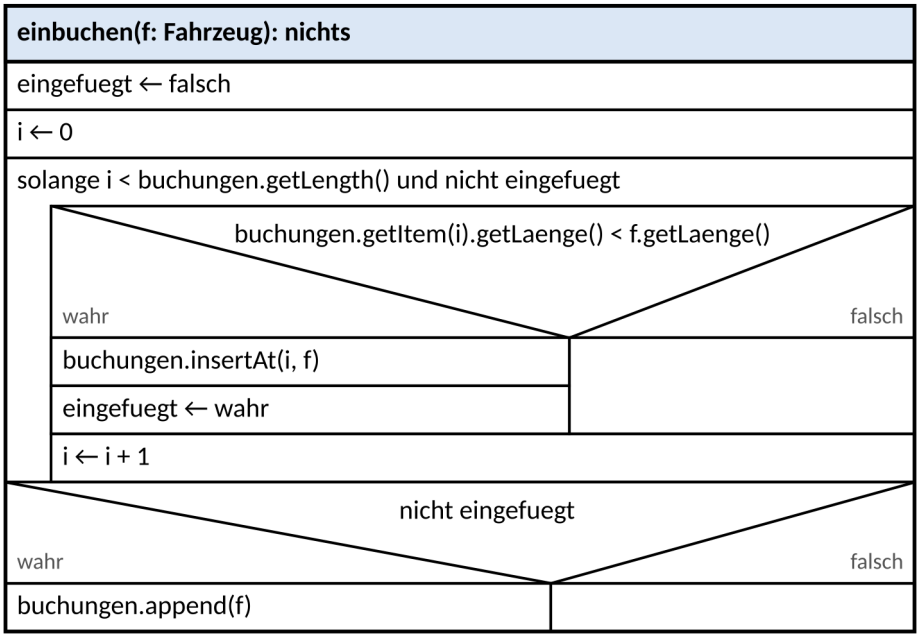


Abbildung EH.3: Alternative mit Wahrheitswert eingefuegt

Typische Fehler

- Bedingungen vertauscht (getItem(i) vor der Index-Prüfung) → Laufzeitfehler, wenn f ans Ende gehört.
- In einer Zählschleife eingefügt, ohne abzubrechen → f mehrfach eingefügt bzw. Endlosschleife (Liste wächst).
- Anhängen vergessen, wenn f das kürzeste Fahrzeug ist.
- Aufsteigend statt absteigend sortiert.
- Java-Syntax (==, ++, Semikolon) statt Struktogramm-Notation.

e) Tracetabelle mit Stapel und Schlange; Funktionsweise erläutern 4 BE (I 1 · II 3 · III 0)

Musterlösung

Tracetabelle (geprüft durch Ausführen einer zeilengetreuen, instrumentierten Java-Fassung von Abbildung 4):

f (Kennzeichen)	anleger (vorn zuerst)	s (oben zuerst)	h (vorn zuerst)
vor der 1. Schleife	HB-R 5, OL-W 8, NH-M 1, DEL-S 6, NH-Z 2	–	–
HB-R 5	OL-W 8, NH-M 1, DEL-S 6, NH-Z 2	–	HB-R 5
OL-W 8	NH-M 1, DEL-S 6, NH-Z 2	OL-W 8	HB-R 5
NH-M 1	DEL-S 6, NH-Z 2	OL-W 8	HB-R 5, NH-M 1
DEL-S 6	NH-Z 2	DEL-S 6, OL-W 8	HB-R 5, NH-M 1
NH-Z 2	–	DEL-S 6, OL-W 8	HB-R 5, NH-M 1, NH-Z 2
nach der 2. Schleife	DEL-S 6, OL-W 8	–	HB-R 5, NH-M 1, NH-Z 2
nach der 3. Schleife	DEL-S 6, OL-W 8, HB-R 5, NH-M 1, NH-Z 2	–	–

Funktionsweise: In der ersten Schleife wird die Schlange anleger vollständig geleert. Jedes Fahrzeug, das länger als grenze ist, wird auf den Stapel s gelegt, alle anderen werden in der Hilfsschlange h angestellt. Danach werden zuerst alle Fahrzeuge aus s und anschließend alle aus h wieder in anleger angestellt. Die langen Fahrzeuge stehen also vorn (sie werden zuerst verladen), die kurzen dahinter. Die kurzen behalten ihre Reihenfolge (Schlange: FIFO), die langen werden umgekehrt (Stapel: LIFO). Alle Entnahmen sind durch die Leer-Abfragen der Schleifen abgesichert.

Änderung: Statt des Stapels s wird eine zweite Hilfsschlange verwendet (s.push(f) → s.enqueue(f), anleger.enqueue(s.pop()) → anleger.enqueue(s.dequeue())). Gleichwertig: die langen Fahrzeuge vor dem Zurückschichten über einen zweiten Stapel noch einmal umdrehen.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Erster Durchlauf und Aufteilung nach grenze (> 10)	I	1	☐
Vollständige Tracetabelle einschließlich 2. und 3. Schleife	II	1	☐
Erläuterung der Funktionsweise (Aufteilen, lange vor kurzen, Umkehrung durch den Stapel)	II	1	☐
Änderung: Schlange statt Stapel für die langen Fahrzeuge	II	1	☐

Alternativen / Teilpunkte: Die Tabelle darf Fahrzeuge mit Länge notieren; Zwischenzeilen können zusammengefasst werden, solange die Zustände nach jedem Durchlauf erkennbar sind. Folgefehler aus der Tabelle werden in der Erläuterung nicht erneut abgezogen.

Typische Fehler

- Stapel s in falscher Reihenfolge notiert (OL-W 8 oben).
- Fahrzeug mit Länge genau 10 bzw. 11 falsch zugeordnet (> grenze).
- anleger in der ersten Schleife nicht verkürzt (dequeue entnimmt!).
- Erläuterung beschreibt nur die Befehle („erst dequeue, dann push …“), nicht die Wirkung.

f) Fehleranalyse: delete in einer Vorwärtsschleife 3 BE (I 1 · II 1 · III 1)

Musterlösung

Ablauf von verladen(7) (geprüft durch Ausführen von Abbildung 5; deck oben zuerst):

i	n	getItem(i)	Aktion	buchungen danach	deck danach (oben zuerst)
0	5	HB-F 4 (17)	–	HB-F 4, OL-C 3, NH-E 8, DEL-G 1, NH-H 6	NH-K 7, DEL-A 2, NH-P 4, OL-B 9, NH-T 3
1	5	OL-C 3 (12)	–	HB-F 4, OL-C 3, NH-E 8, DEL-G 1, NH-H 6	NH-K 7, DEL-A 2, NH-P 4, OL-B 9, NH-T 3
2	5	NH-E 8 (7)	push, delete(2)	HB-F 4, OL-C 3, DEL-G 1, NH-H 6	NH-E 8, NH-K 7, DEL-A 2, NH-P 4, OL-B 9, NH-T 3
3	5	NH-H 6 (4)	push, delete(3)	HB-F 4, OL-C 3, DEL-G 1	NH-H 6, NH-E 8, NH-K 7, DEL-A 2, NH-P 4, OL-B 9, NH-T 3
4	5	Länge 3!	Laufzeitfehler	(IndexOutOfBounds-Exception)	Abbruch

Ursache: n = 5 wird vor der Schleife einmal festgelegt. Nach `buchungen.delete(i)` rücken alle folgenden Fahrzeuge um eine Position nach vorn, die Liste wird kürzer – die Schleife erhöht i aber trotzdem. **Auswirkungen:** (1) Das Fahrzeug, das auf Position i nachrückt, wird übersprungen: DEL-G 1 (7 m) wird nicht verladen. (2) Die Schleife läuft bis i = 4, obwohl die Liste nur noch 3 Elemente hat: getItem(4) → Laufzeitfehler (IndexOutOfBoundsException). NH-E 8 und NH-H 6 stehen danach bereits auf dem Deck, die Methode liefert keinen Wert. (Bei verladen(4) tritt kein Absturz auf, weil nur das letzte Fahrzeug gelöscht wird – der Fehler zeigt sich nur bei bestimmten Belegungen; geprüft.)

Korrektur (geprüft: verladen(7) liefert 3, buchungen = HB-F 4, OL-C 3, deck oben NH-H 6, DEL-G 1, NH-E 8, …; 4000 Zufallsfälle):

```
public int verladen(int maxLaenge) {
    int anzahl = 0;
    int i = 0;
    while (i < buchungen.getLength()) {
        Fahrzeug f = (Fahrzeug) buchungen.getItem(i);
        if (f.getLaenge() <= maxLaenge) {
            deck.push(f);
            buchungen.delete(i);
            anzahl++;
        } else {
            i++;
        }
    }
    return anzahl;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Fehlerstelle: delete in der Vorwärtsschleife mit festem n	I	1	☐
Ablauf: DEL-G 1 übersprungen, getItem(4) bei Länge 3 → Laufzeitfehler	II	1	☐
Korrektur mit Begründung (Länge neu abfragen, i nur ohne Löschen erhöhen; Reihenfolge bleibt)	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: for-Schleife mit `i < buchungen.getLength()` und `i--` nach dem delete. Eine rückwärts laufende Schleife vermeidet den Index-Fehler, legt die Fahrzeuge aber in umgekehrter Reihenfolge auf das Deck (geprüft: NH-E 8 statt NH-H 6 oben) und verletzt damit die Vorgabe „in der Reihenfolge der Liste“ – mit Hinweis darauf voll, ohne Hinweis höchstens 1 von 2 BE für Ablauf/Korrektur. Nur n durch `buchungen.getLength()` zu ersetzen beseitigt den Absturz, nicht das Überspringen: kein Korrekturpunkt.

Typische Fehler

- Nur den Absturz erkannt, das Überspringen von DEL-G 1 übersehen.
- „< statt <=“ als Fehler vermutet – die Grenze n ist korrekt, nur nicht mehr aktuell.
- Korrektur nur durch Ersetzen von n durch getLength() (Überspringen bleibt).
- Rückwärtsschleife ohne Beachtung der Deck-Reihenfolge.

Anhang: Papier-Compiler-Workspace zu c)

Für den Papier-Compiler legst du in der Online-IDE einen Workspace „W4 Boss 4 – Papier-Compiler“ mit den Klassen Fahrzeug und Faehre (Gerüst unten, ausfahrtBis mit leerem Rumpf und return 0;) und der Testklasse BossTest an, aktivierst die Bibliothek „Niedersachsen“ (Rechtsklick auf den Workspace → Einstellungen → Häkchen) und verteilst ihn über „An Klasse austeilen“. Die Lernenden tippen ihre Handschrift **unverändert** in den Rumpf ab und starten die Tests im Reiter „Testrunner“. Erst danach dürfen Fehler korrigiert werden (dann im Antwortbogen „grün nach Korrektur“ ankreuzen). einbuchen, umsortieren und verladen werden für den Papier-Compiler nicht benötigt.

Vor Einsatz prüfen

Bibliothek „Niedersachsen“ im Workspace aktivieren – ohne sie sind Stack und Queue unbekannt (java.util-Klassen sind dann ausgeblendet). Casts (Fahrzeug) deck.pop() im Dialekt der Online-IDE und den JUnit-Dialekt (parameterloser Konstruktor der Testklasse, @Test, globale assertEquals/assertTrue mit Meldung als letztem Argument) vor dem ersten Einsatz mit einem Probedurchlauf testen – insbesondere den Objektvergleich `n.abfahren() != f[i]` in der privaten Hilfsmethode deckUnveraendert und die Meldung des Testrunners bei einer IllegalStateException (Mutante „keine Leer-Abfrage“): Der Testrunner muss danach weiterlaufen. Die Hilfsmethoden nebelstern() und deckUnveraendert(…) sind private und ohne @Test.

Klasse Fahrzeug (vorgegeben)

```
public class Fahrzeug {

    private String kennzeichen;
    private int laenge;

    public Fahrzeug(String kennzeichen, int laenge) {
        this.kennzeichen = kennzeichen;
        this.laenge = laenge;
    }

    public String getKennzeichen() {
        return kennzeichen;
    }

    public int getLaenge() {
        return laenge;
    }
}
```

Klasse Faehre (Gerüst für den Workspace; Konstruktor, auffahren und abfahren sind vorhanden)

```
public class Faehre {

    private String name;
    private Stack deck;          // Stapel vom Inhaltstyp Fahrzeug
    private Queue anleger;       // Schlange vom Inhaltstyp Fahrzeug
    private DynArray buchungen;  // dynamische Reihung vom Inhaltstyp Fahrzeug

    public Faehre(String name) {
        this.name = name;
        deck = new Stack();
        anleger = new Queue();
        buchungen = new DynArray();
    }

    public void auffahren(Fahrzeug f) {
```

```
        deck.push(f);
    }

    public Fahrzeug abfahren() {
        if (deck.isEmpty()) {
            return null;
        }
        return (Fahrzeug) deck.pop();
    }

    public int ausfahrtBis(String kennzeichen) {
        // HIER die Handschrift aus Aufgabe c) UNVERAENDERT abtippen
        return 0;
    }
}
```

Testklasse BossTest (8 Tests; mit der Musterlösung und beiden Alternativen alle grün, typische Fehler werden erkannt – geprüft mit javac und Nachbauten der Bibliotheksklassen, siehe Tabelle)

```
public class BossTest {

    private Fahrzeug[] f;

    public BossTest() {
    }

    // Deck aus Abbildung 2: f[0] faehrt zuerst auf (ganz unten), f[4] steht oben
    private Faehre nebelstern() {
        f = new Fahrzeug[5];
        f[0] = new Fahrzeug("NH-T 3", 7);
        f[1] = new Fahrzeug("OL-B 9", 18);
        f[2] = new Fahrzeug("NH-P 4", 4);
        f[3] = new Fahrzeug("DEL-A 2", 12);
        f[4] = new Fahrzeug("NH-K 7", 5);
        Faehre n = new Faehre("Nebelstern");
        for (int i = 0; i < 5; i++) {
            n.auffahren(f[i]);
        }
        return n;
    }

    // prueft, ob das Deck noch genau f[4], f[3], ..., f[0] (oben zuerst) enthaelt
    private boolean deckUnveraendert(Faehre n) {
        for (int i = 4; i >= 0; i--) {
            if (n.abfahren() != f[i]) {
                return false;
            }
        }
        return n.abfahren() == null;
    }

    @Test
    public void beispiel() {
        Faehre n = nebelstern();
        assertEquals(3, n.ausfahrtBis("OL-B 9"), "vor OL-B 9 fahren 3 ab");
    }

    @Test
    public void oberstesFahrzeug() {
```

```
        Faehre n = nebelstern();
        assertEquals(0, n.ausfahrtBis("NH-K 7"), "NH-K 7 steht oben");
    }

    @Test
    public void unterstesFahrzeug() {
        Faehre n = nebelstern();
        assertEquals(4, n.ausfahrtBis("NH-T 3"), "NH-T 3 steht ganz unten");
    }

    @Test
    public void nichtAnBord() {
        Faehre n = nebelstern();
        assertEquals(-1, n.ausfahrtBis("HB-X 1"), "nicht an Bord liefert -1");
    }

    @Test
    public void leeresDeck() {
        Faehre n = new Faehre("Leer");
        assertEquals(-1, n.ausfahrtBis("NH-K 7"), "leeres Deck liefert -1");
        assertTrue(n.abfahren() == null, "Deck bleibt leer");
    }

    @Test
    public void unveraendertNachTreffer() {
        Faehre n = nebelstern();
        n.ausfahrtBis("OL-B 9");
        assertTrue(deckUnveraendert(n), "Deck nach Treffer unveraendert");
    }

    @Test
    public void unveraendertOhneTreffer() {
        Faehre n = nebelstern();
        n.ausfahrtBis("HB-X 1");
        assertTrue(deckUnveraendert(n), "Deck ohne Treffer unveraendert");
    }

    @Test
    public void mehrereAufrufe() {
        Faehre n = nebelstern();
        assertEquals(2, n.ausfahrtBis("NH-P 4"), "erster Aufruf");
        assertEquals(2, n.ausfahrtBis("NH-P 4"), "zweiter Aufruf liefert dasselbe");
        assertEquals(1, n.ausfahrtBis("DEL-A 2"), "dritter Aufruf");
        assertTrue(deckUnveraendert(n), "Deck nach drei Aufrufen unveraendert");
    }
}
```

Kontrolle der Testklasse mit typischen Fehlvarianten (Mutanten)

Variante von ausfahrtBis	Tests grün
Musterlösung (Hilfsstapel, Abbruch beim Treffer, Ergebnisvariable)	8/8
Alternative (1): top() prüfen, nur Fahrzeuge darüber umschichten, Wahrheitswert gefunden	8/8
Alternative (2): ganzen Stapel umschichten, erstes Vorkommen merken	8/8
keine Leer-Abfrage in der Schleife (pop auf leerem Stapel)	5/8
return beim Treffer, Deck nicht wiederhergestellt	6/8
Hilfsschlange statt Hilfsstapel (Reihenfolge umgekehrt)	5/8

Variante von ausfahrtBis	Tests grün
Zurückschichten fehlt	5/8
gesuchtes Fahrzeug mitgezählt (anzahl + 1)	4/8
nicht gefunden: Anzahl aller statt −1	6/8
Schleife ohne Abbruch, Ergebnis wird von tieferen Fahrzeugen überschrieben	5/8
leerer Rumpf (Gerüst, return 0;)	3/8